

ADR-05: Patrones GOF integrados en ZenFlow

ZenFlow — Singleton para control de instancias de repositorios y Proxy como guardián entre la API y la lógica de negocio

Campo	Valor
Autor	Dylan Vazquez
Fecha	26/06/2026
Estado	Propuesto

Contexto

Los ADR-01 al ADR-04 documentaron las decisiones estructurales de ZenFlow: tecnología (WPF con arquitectura en capas), persistencia (JSON a través de interfaces repositorio), y la exposición de datos mediante una API REST. Ninguno de esos ADR formaliza cómo se controla la creación y el acceso a los objetos que tocan los archivos de datos, ni cómo se agregan responsabilidades transversales — seguridad y logging — sin contaminar la lógica de negocio existente.

Al integrar la API REST de ADR-04 sobre el mismo sistema WPF, surgieron dos problemas concretos que los patrones GOF resuelven:

- Problema de instancias múltiples sobre archivos compartidos: `TareaRepoJson` y `HabitoRepoJson` leen y escriben sobre `tareas.json` y `habitos.json`. Si la app WPF y la API corren al mismo tiempo y cada una crea su propia instancia del repositorio, dos operaciones de escritura simultáneas pueden corromper el archivo JSON. ADR-04 ya documentó esto como una limitación técnica explícita. La solución arquitectónica es garantizar que exista una única instancia de cada repositorio en todo el proceso, con un mecanismo de bloqueo que sincronice el acceso concurrente.
- Problema de responsabilidades transversales en la API: ADR-04 documentó como deuda técnica que la API no tiene autenticación y que cualquiera con la URL puede leer y modificar datos. Agregar verificación de seguridad y registro de operaciones directamente dentro de `GestorTareas` o `GestorHabitos` viola el SRP — esos gestores tienen una única responsabilidad (la lógica de negocio) y no deben conocer detalles de seguridad ni de logging. Se necesita una capa intermedia que intercepte las llamadas antes de que lleguen al gestor.

Decisión

Integrar dos patrones GOF de categorías distintas para resolver los problemas identificados:

- Singleton (Creacional): aplicado sobre `TareaRepoJson` y `HabitoRepoJson` para garantizar una única instancia de cada repositorio durante el ciclo de vida de la aplicación, con un bloque `lock` interno que sincroniza el acceso concurrente de WPF y la API sobre los mismos archivos JSON.
- Proxy (Estructural): implementado como `TareaServiceProxy` y `HabitoServiceProxy`, clases que implementan las mismas interfaces de lógica que los gestores reales. El Proxy intercepta cada operación, ejecuta las responsabilidades transversales

(verificación de autenticación y registro en bitácora), y delega al gestor real únicamente si la operación es válida.

Singleton — instancia única de repositorios con sincronización

El problema concreto que Singleton resuelve está en App.xaml.cs. Hoy el código es:

```
GestorTareas = new GestorTareas(new TareaRepoJson());
GestorHabitos = new GestorHabitos(new HabitoRepoJson());
```

Esto funciona mientras solo WPF use los repositorios. Pero la API de ADR-04 también necesita TareaRepoJson y HabitoRepoJson. Si cada proyecto crea su propia instancia, las escrituras simultáneas sobre el mismo archivo JSON no tienen coordinación — es la condición de carrera documentada en ADR-04.

La implementación aplica el patrón Singleton con double-check locking, el mecanismo estándar para entornos con múltiples hilos en C#:

```
public class TareaRepoJson : ITareaRepo
{
    // Singleton: instancia única y lock para concurrencia
    private static TareaRepoJson? _instancia;
    private static readonly object _lock = new object();

    private TareaRepoJson() { }

    public static TareaRepoJson Instancia
    {
        get
        {
            if (_instancia == null)
            {
                lock (_lock)
                {
                    if (_instancia == null)
                        _instancia = new TareaRepoJson();
                }
            }
            return _instancia;
        }
    }

    // Los métodos Guardar(), ObtenerTodas() y Eliminar()
    // también usan lock(_lock) internamente al escribir.
}
```

El constructor privado impide que cualquier otra clase use new TareaRepoJson(). App.xaml.cs y los controllers de la API consumen la misma instancia a través de TareaRepoJson.Instancia, y el lock interno garantiza que si WPF y la API escriben al mismo tiempo, las operaciones se serialicen en lugar de sobreponerse.

La razón por la que Singleton se aplica a los repositorios y no a los gestores es que el problema de concurrencia ocurre en la capa de datos — es donde se toca el archivo físico. GestorTareas puede tener múltiples instancias sin problema porque no accede al disco directamente; delega esa responsabilidad al repositorio.

Proxy — guardián entre la API y la lógica de negocio

El Proxy implementa la misma interfaz que el objeto real y actúa como intermediario. En ZenFlow, TareaServiceProxy implementa la misma interfaz que GestorTareas (o una interfaz IGestorTareas extraída para este propósito), lo que permite que los controllers de la API lo usen sin saber si están hablando con el gestor real o con el proxy:

```
public class TareaServiceProxy : IGestorTareas
{
    private readonly GestorTareas _gestor;
    private readonly string _tokenEsperado = "zenflow-token-2025";

    public TareaServiceProxy(GestorTareas gestor)
    {
        _gestor = gestor;
    }

    public void AgregarTarea(string titulo, DateTime fechaLimite, string token)
    {
        // 1. Verificación de autenticación
        if (token != _tokenEsperado)
            throw new UnauthorizedAccessException("Token inválido.");

        // 2. Registro en bitácora
        File.AppendAllText("bitacora.log",
            $"[{DateTime.Now}] AgregarTarea: {titulo}\n");

        // 3. Delegación al gestor real
        _gestor.AgregarTarea(titulo, fechaLimite);
    }
}
```

Los controllers de la API reciben IGestorTareas por inyección de dependencias. En producción reciben TareaServiceProxy; en pruebas pueden recibir un mock o el gestor real directamente. GestorTareas no sabe que existe el Proxy y no cambia en absoluto.

La razón por la que Proxy resuelve el problema mejor que modificar GestorTareas directamente: agregar autenticación y logging dentro de GestorTareas violaría el SRP — ese gestor tiene una responsabilidad (la lógica de tareas) y no debe conocer detalles de seguridad ni de infraestructura de logging. El Proxy mantiene esas responsabilidades separadas y hace que GestorTareas sea reutilizable tanto desde WPF (sin proxy, sin autenticación necesaria) como desde la API (con proxy, con todas las verificaciones).

Alternativas consideradas

Alternativa	Por qué la descarté
Multiton (variante de Singleton)	Permite múltiples instancias indexadas por clave. No aplica aquí porque para ZenFlow solo existe un archivo tareas.json y uno habitos.json — no hay múltiples fuentes del mismo tipo que administrar.
Object Pool (Creacional)	Reutiliza un conjunto fijo de instancias en lugar de crearlas y destruirlas. Útil para objetos costosos de construir como conexiones de

	base de datos. TareaRepoJson es liviano — no tiene costo de inicialización. Singleton es suficiente.
Decorator (Estructural)	Agrega responsabilidades a un objeto envolviéndolo. Podría usarse para agregar logging sobre el gestor, pero requiere heredar o implementar la misma interfaz y componer en capas. Proxy es más directo para interceptar y controlar acceso porque el propósito del patrón es exactamente ese: ser un sustituto con control.
Facade (Estructural)	Simplifica una interfaz compleja exponiendo solo lo necesario. Útil cuando hay muchos subsistemas. En ZenFlow los gestores ya son simples y su interfaz ya está reducida al mínimo necesario. Facade no agrega valor sobre lo que ya existe.
Chain of Responsibility (Comportamiento)	Encadena manejadores para procesar una petición secuencialmente. Sería una alternativa para el pipeline de validación, pero implica más clases y configuración que un Proxy simple. Para dos verificaciones (autenticación + log), la cadena agrega complejidad sin beneficio.

Consecuencias

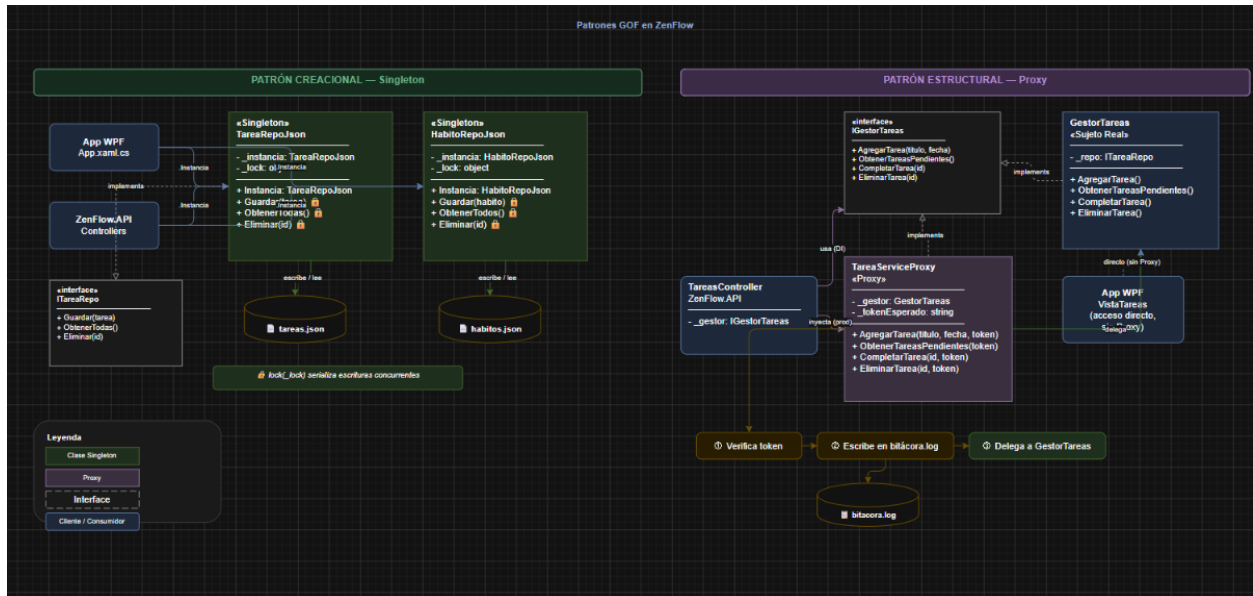
Lo que gano:

- Técnica (Singleton): la condición de carrera documentada en ADR-04 entre WPF y la API sobre los archivos JSON queda controlada. El lock garantiza que las escrituras concurrentes se serialicen. Una sola instancia también significa que si en el futuro se agrega un mecanismo de caché en memoria sobre los repositorios, todos los consumidores comparten el mismo caché automáticamente.
- Técnica (Proxy): GestorTareas y GestorHabitos no cambian una sola línea para soportar autenticación y logging en la API. Esas responsabilidades viven en una clase dedicada. Si mañana el mecanismo de autenticación cambia de token estático a JWT, solo cambia TareaServiceProxy, no la lógica de negocio.
- Proceso: el Proxy cancela explícitamente la deuda técnica de 'sin autenticación' documentada en ADR-04, y Singleton cancela la limitación técnica de 'condiciones de carrera en escrituras simultáneas'. Ambas consecuencias negativas de ADR-04 tienen ahora una respuesta arquitectónica documentada.

Lo que sacrifico o asumo:

- Limitación técnica (Singleton): una instancia única significa que si en el futuro se necesitan dos bases de datos del mismo tipo en paralelo — por ejemplo, una de prueba y una de producción en el mismo proceso — el Singleton lo impide sin refactorización. Para ZenFlow en su estado actual esto no es un escenario real.
- Deuda técnica (Singleton): el Singleton estático es difícil de reemplazar por un mock en pruebas unitarias de los repositorios mismos. Si se agrega una suite de tests formales, habría que considerar un mecanismo de reset de instancia para tests o migrar a DI con ciclo de vida Singleton gestionado por el contenedor.
- Deuda técnica (Proxy): el token estático hardcodeado en TareaServiceProxy es un punto de partida funcional, pero no es seguro para exposición pública. Si la API de ZenFlow se expone fuera de la red local, habría que migrar a JWT u OAuth como mecanismo de autenticación real, lo que requeriría modificar el Proxy pero no el resto del sistema.

Diagrama



Declaración de uso de IA

Este ADR fue guiado y contribuido por la ayuda de Claude en el consejo para tomar una buena decisión y poder mantener el rumbo del proyecto de manera adecuada y correcta para el buen desarrollo del proyecto-